Competitive Programming Reference Sheet

AbdElrhman Mohamed

Contents

1 Number Theory

GCD, LCM

```
// A x B = GCD(A, B) x LCM(A, B)
// LCM(A, B) = (A x B) / GCD(A, B)
long long lcm(long long a, long long b) {
    return (a / std::gcd(a, b)) * b;
}
long long gcd(long long a, long long b) {
    return b == 0 ? a : gcd(b, a % b);
}
```

Prime Factorization (map)

```
// 360 = 2 x 2 x 2 x 3 x 3 x 5
// 360 = pow(2, 3) x pow(3, 2) x pow(5, 1)
// to build a valid divisor of 360, we choose primes
// and multiply them together.
// Total divisors = (3+1) x (2+1) x (1+1) = 4*3*2 = 24
// from the first number (2) we can choose 0..3 = 4 choices
// = power + 1
map<long long, long long> prime_power;
void factorize(long long n) {
    prime_power.clear(); // clear old data for multiple tests
    for (long long i = 2; (long long)(i * i) <= n; i++) {
        int count = 0;
        while (n % i == 0) {
            count += 1;
            n /= i;
        }
        if (count > 0) {
            prime_power[i] = count;
        }
    }
    if (n > 1) {
        prime_power[n] = 1;
    }
}
```

Smallest Prime Factor Sieve

```
const int MAXN = 1e7;
vector<int> spf(MAXN + 1);
void gen_spf() {
    // Initialize every number as its own smallest prime factor
    for (int i = 0; i <= MAXN; ++i) {
        spf[i] = i;
    }
    // build spf using sieve:
    for (long long i = 2; i * i <= MAXN; ++i) {
        // if i is still prime (not converted by a smaller number)
        // then mark all its multiples as not prime.
        // If spf[i] == i, then i is a prime number
        if (spf[i] == i) {
            for (long long j = (i * i); j <= MAXN; j += i) {
                // make sure the spf was not processed before
                if (spf[j] == j) {
                    spf[j] = i; // spf of all multiples is this number
                }
            }
```

```
        }
    }
}
```

Sieve of Eratosthenes

```
const int N = 1000000;
vector<bool> is_prime(N + 1, true);
void sieve() {
    is_prime[0] = is_prime[1] = false;
    for (int i = 2; (long long)(i * i) <= N; i++) {
        if (is_prime[i]) {
            for (int j = (long long)(i * i); j <= N; j += i) {
                is_prime[j] = false;
            }
        }
    }
}
```

Sieve with Prime List

```
const int N = 1000000;
vector<bool> is_prime(N + 1, true);
vector<int> prime_numbers;
void sieve() {
    is_prime[0] = is_prime[1] = false;
    for (int i = 2; i <= N; i++) {
        if (is_prime[i]) {
            prime_numbers.push_back(i);
            if (1LL * i * i <= N) {
                for (int j = i * i; j <= N; j += i)
                    is_prime[j] = false;
            }
        }
    }
}
```

2 Arrays

Kadane's Algorithm (Max Subarray Sum)

```
long long maxSubarraySum(vector<int>& a) {
    long long best = a[0], cur = a[0];
    for (int i = 1; i < a.size(); i++) {
        cur = max((long long)a[i], cur + a[i]);
        best = max(best, cur);
    }
    return best;
}
```

Kadane's with Indices

```
// a custom version for getting the subarray l and r too!
tuple<int,int,int> maxSubarraySum(vector<int> &arr) {
    int n = arr.size();
    int maxEnding = arr[0];
    int res = arr[0];
    int cur_l = 0;
    int best_l = 0, best_r = 0;
```

```
    for (int i = 1; i < n; i++) {
        if (arr[i] > maxEnding + arr[i]) {
            maxEnding = arr[i];
            cur_l = i;
        } else {
            maxEnding += arr[i];
        }
        if (maxEnding > res) {
            res = maxEnding;
            best_l = cur_l;
            best_r = i;
        }
    }
    return {res, best_l, best_r};
}
```

3 Graphs

BFS

```
void bfs(const vector<vector<int>>& adj, int u) {
    queue<int> q;
    vector<bool> vis(adj.size(), false);
    q.push(u);
    vis[u] = true;
    while (!q.empty()) {
        int curr = q.front();
        q.pop();
        cout << curr << '\n';
        for (auto child : adj[curr]) {
            if (!vis[child]) {
                vis[child] = true;
                q.push(child);
            }
        }
    }
}
```

DFS – Cycle Detection & Topological Order

```
vector<vector<int>> adj;
vector<int> visited; // 0=unvisited, 1=visiting, 2=fully visited
vector<int> order;
bool has_cycle = false;
void dfs(int u) {
    visited[u] = 1;
    for (int v : adj[u]) {
        if (visited[v] == 1) {
            has_cycle = true;
        } else if (visited[v] == 0) {
            dfs(v);
        }
    }
    visited[u] = 2;
    order.push_back(u);
}
```

Topological Sort (Kahn's Algorithm)

```
// Kahn's Algorithm (BFS-based): produces a linear ordering of
```

```cpp
// the nodes of a DAG such that for every directed edge u -> v,
// u appears before v in the ordering.
// Time complexity: O(V + E)
// If order.size() < n after running, the graph has a cycle.
vector<int> topoSort(const vector<vector<int>>& adj, int n) {
    vector<int> indeg(n, 0);
    for (int u = 0; u < n; u++)
        for (int v : adj[u])
            indeg[v]++;
    queue<int> q;
    for (int u = 0; u < n; u++)
        if (indeg[u] == 0)
            q.push(u);
    vector<int> order;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        order.push_back(u);
        for (int v : adj[u]) {
            if (--indeg[v] == 0) {
                q.push(v);
            }
        }
    }
    return order; // order.size() < n => graph has a cycle
}
```

DFS – Iterative (Stack)

```cpp
void dfs(vector<vector<int>>& adj, int u) {
    vector<bool> vis(adj.size(), false);
    stack<int> st;
    st.push(u);
    vis[u] = true;
    while (!st.empty()) {
        int curr = st.top();
        st.pop();
        cout << curr << ' ';
        for (int i = (int)adj[curr].size() - 1; i >= 0; --i) {
            int child = adj[curr][i];
            if (!vis[child]) {
                st.push(child);
                vis[child] = true;
            }
        }
    }
}
```

DFS – Recursive

```cpp
void dfs(vector<vector<int>>& adj, int u, vector<bool>& vis) {
    cout << u << ' ';
    vis[u] = true;
    for (auto& c : adj[u]) {
        if (!vis[c]) {
            dfs(adj, c, vis);
        }
    }
}
```

Dijkstra's Algorithm

```
void Dijkstra(const vector<vector<pair<int,int>>>& adj,
              int source, vector<long long>& dist) {
    int n = adj.size();
    dist.assign(n, LLONG_MAX);
    priority_queue<pair<long long,int>,
                   vector<pair<long long,int>>,
                   greater<pair<long long,int>>> min_heap;
    dist[source] = 0;
    min_heap.push({0, source});
    while (!min_heap.empty()) {
        auto [d, u] = min_heap.top();
        min_heap.pop();
        if (d > dist[u]) continue;
        for (auto &[v, w] : adj[u]) {
            if (d + w < dist[v]) {
                dist[v] = d + w;
                min_heap.push({dist[v], v});
            }
        }
    }
}
```

Bellman-Ford Algorithm

```
// Bellman-Ford: single-source shortest paths, works with
// NEGATIVE edge weights. Also detects negative-weight cycles
// reachable from the source.
// Time complexity: O(V * E) -- slower than Dijkstra, only use
// it when you actually have negative weights (otherwise prefer
// Dijkstra: O(E log V)).
//
// How to use:
// 1. Build adj[u] = list of (v, weight) pairs.
// 2. Call bellmanFord(adj, source, n, dist).
// 3. Returns false if a negative cycle is reachable from source
//    (dist[] is then not meaningful); returns true and fills
//    dist[] otherwise.
bool bellmanFord(const vector<vector<pair<int,int>>>& adj,
                 int source, int n, vector<long long>& dist) {
    dist.assign(n, LLONG_MAX);
    dist[source] = 0;
    // Relax all edges (n - 1) times
    for (int i = 0; i < n - 1; i++) {
        for (int u = 0; u < n; u++) {
            if (dist[u] == LLONG_MAX) continue;
            for (auto& [v, w] : adj[u]) {
                if (dist[u] + w < dist[v]) {
                    dist[v] = dist[u] + w;
                }
            }
        }
    }
    // One extra pass: if we can still relax, a negative
    // cycle exists
    for (int u = 0; u < n; u++) {
        if (dist[u] == LLONG_MAX) continue;
        for (auto& [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                return false; // negative cycle reachable
```

```
            }
        }
    }
    return true;
}

// Example usage: reading a weighted directed graph into an
// adjacency list and running Bellman-Ford from a source.
int main() {
    int n, m, source;
    cin >> n >> m >> source; // n = #nodes, m = #edges
    vector<vector<pair<int,int>>> adj(n);
    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        adj[u].push_back({v, w}); // directed edge u -> v
    }
    vector<long long> dist;
    bool noNegativeCycle = bellmanFord(adj, source, n, dist);
    if (!noNegativeCycle) {
        cout << "Negative␣cycle␣detected\n";
    } else {
        for (int i = 0; i < n; i++) {
            cout << "dist[" << i << "]␣=␣"
                 << (dist[i] == LLONG_MAX ? -1 : dist[i]) << '\n';
        }
    }
    return 0;
}
```

Floyd-Warshall Algorithm

```
// Floyd-Warshall: shortest paths between ALL pairs of nodes.
// Works with negative edge weights, but NOT with negative
// cycles.
// Time complexity: O(V^3) -- only practical for small graphs
// (roughly V <= 400-500).
//
// How to use:
// 1. Build dist[u][v] = weight of the direct edge u -> v
//    (INF if no direct edge, 0 when u == v).
// 2. Call floydWarshall(dist, n) -- updates dist[][] in place.
// 3. Afterwards dist[u][v] holds the shortest path from u to v
//    (INF if still unreachable).
const long long INF = LLONG_MAX / 2; // halved to avoid overflow
void floydWarshall(vector<vector<long long>>& dist, int n) {
    for (int k = 0; k < n; k++) {       // intermediate node
        for (int u = 0; u < n; u++) {   // source
            for (int v = 0; v < n; v++) { // destination
                if (dist[u][k] + dist[k][v] < dist[u][v]) {
                    dist[u][v] = dist[u][k] + dist[k][v];
                }
            }
        }
    }
}

// Example usage: reading a weighted directed graph (as edges)
// and turning it into the matrix Floyd-Warshall needs.
int main() {
```

```
    int n, m;
    cin >> n >> m; // n = #nodes, m = #edges
    vector<vector<long long>> dist(n, vector<long long>(n, INF));
    for (int i = 0; i < n; i++) dist[i][i] = 0;
    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        dist[u][v] = min(dist[u][v], (long long)w); // multi-edges
    }
    floydWarshall(dist, n);
    for (int u = 0; u < n; u++) {
        for (int v = 0; v < n; v++) {
            cout << (dist[u][v] >= INF ? -1 : dist[u][v]) << ' ';
        }
        cout << '\n';
    }
    return 0;
}
```

Lexicographical BFS

```
void lexicographicalBfs(vector<vector<int>>& adj,
                        vector<bool>& vis, int src) {
    priority_queue<int, vector<int>, greater<int>> mh;
    mh.push(src);
    vis[src] = true;
    while (!mh.empty()) {
        int u = mh.top();
        mh.pop();
        cout << u << "␣";
        for (auto& v : adj[u]) {
            if (!vis[v]) {
                mh.push(v);
                vis[v] = true;
            }
        }
    }
}
```

4 PBDS

Ordered Set

```
#include <bits/stdc++.h>
using namespace std;
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

template <typename T>
using ordered_set = tree<T, null_type, less<T>,
      rb_tree_tag, tree_order_statistics_node_update>;
```

Ordered Multiset

```
#include <bits/stdc++.h>
using namespace std;
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
```

```
template <typename T>
using ordered_multiset = tree<pair<T,int>, null_type,
      less<pair<T,int>>, rb_tree_tag,
      tree_order_statistics_node_update>;
```

5 Segment Trees

Range Sum, Point Update

```
#include <bits/stdc++.h>
using namespace std;
#define _42 ios_base::sync_with_stdio(false); \
    cin.tie(NULL); cout.tie(NULL);

class SegmentTree {
private:
    vector<long long> tree;
    vector<long long> arr;
    int n;

    // Build the tree
    void build(int node, int l, int r) {
        if (l == r) {
            tree[node] = arr[l];
            return;
        }
        int mid = (l + r) / 2;
        build(node * 2 + 1, l, mid);
        build(node * 2 + 2, mid + 1, r);
        tree[node] = tree[node*2+1] + tree[node*2+2];
    }

    // Query sum in [ql, qr]
    long long query(int node, int l, int r, int ql, int qr) {
        if (r < ql || l > qr) return 0;          // no overlap
        if (ql <= l && r <= qr) return tree[node]; // full overlap
        int mid = (l + r) / 2;                    // partial
        return query(node*2+1, l, mid, ql, qr)
             + query(node*2+2, mid+1, r, ql, qr);
    }

    // Update arr[idx] = value
    void update(int node, int l, int r, int idx, long long value) {
        if (l == r) {              // leaf node
            tree[node] = value;
            arr[idx] = value;
            return;
        }
        int mid = (l + r) / 2;
        if (idx <= mid)
            update(node*2+1, l, mid, idx, value);
        else
            update(node*2+2, mid+1, r, idx, value);
        tree[node] = tree[node*2+1] + tree[node*2+2];
    }

public:
    SegmentTree(vector<long long>& a) {
        arr = a;
```

```
        n = a.size();
        tree.assign(4 * n, 0);
        build(0, 0, n - 1);
    }
    long long query(int l, int r) {
        return query(0, 0, n - 1, l, r);
    }
    void update(int idx, long long value) {
        update(0, 0, n - 1, idx, value);
    }
};

void solve() {
    int n, q;
    cin >> n >> q;
    vector<long long> array(n);
    for (int i = 0; i < n; i++) cin >> array[i];
    SegmentTree segmentTree(array);
    while (q--) {
        int operation;
        cin >> operation;
        if (operation == 1) {
            // point update goes here
        } else {
            int left, right;
            cin >> left >> right;
            cout << segmentTree.query(left, right - 1) << '\n';
        }
    }
}

int main() {
    _42
    int t = 1; // cin >> t;
    while (t--) solve();
    return 0;
}
```

Maximum Subarray Sum (Segment Tree)

```
#include <bits/stdc++.h>
using namespace std;
#define _42 ios_base::sync_with_stdio(false); \
    cin.tie(NULL); cout.tie(NULL);

struct Node {
    long long sum, pref, suff, ans;
};

Node make_node(int val, bool allow_empty = true) {
    int sum = val;
    val = allow_empty ? max(0, val) : val;
    return {sum, val, val, val};
}

Node combine(Node left, Node right) {
    Node res;
    res.sum = left.sum + right.sum;
    res.pref = max(left.pref, left.sum + right.pref);
    res.suff = max(right.suff, right.sum + left.suff);
```

```
    res.ans = max({left.ans, right.ans,
                   left.suff + right.pref});
    return res;
}

class SegmentTree {
private:
    vector<Node> tree;
    vector<int> a;

    void build(int node, int left, int right) {
        if (left == right) {              // leaf node
            tree[node] = make_node(a[left]);
            return;
        }
        int mid = (left + right) / 2;
        build(node * 2 + 1, left, mid);       // left child
        build(node * 2 + 2, mid + 1, right);  // right child
        Node leftNode = tree[node * 2 + 1];
        Node rightNode = tree[node * 2 + 2];
        tree[node] = combine(leftNode, rightNode);
    }

    Node query(int node, int left, int right,
               int qLeft, int qRight) {
        if (qLeft <= left && right <= qRight)
            return tree[node];
        int mid = (left + right) / 2;
        if (qRight <= mid)
            return query(node * 2 + 1, left, mid, qLeft, qRight);
        if (qLeft > mid)
            return query(node*2+2, mid+1, right, qLeft, qRight);
        Node leftNode = query(node*2+1, left, mid, qLeft, mid);
        Node rightNode = query(node*2+2, mid+1, right,
                               mid+1, qRight);
        return combine(leftNode, rightNode);
    }

    void update(int node, int left, int right,
                int index, int value) {
        if (left == right) {          // leaf node
            a[index] = value;
            tree[node] = make_node(value);
            return;
        }
        int mid = (left + right) / 2;
        if (index <= mid)
            update(node*2+1, left, mid, index, value);
        else
            update(node*2+2, mid+1, right, index, value);
        Node leftNode = tree[node * 2 + 1];
        Node rightNode = tree[node * 2 + 2];
        tree[node] = combine(leftNode, rightNode);
    }

public:
    SegmentTree(vector<int>& a) {
        this->a = a;
        tree.resize(4 * a.size());
        build(0, 0, a.size() - 1);
    }
```

```
    long long query(int left, int right) {
        Node node = query(0, 0, a.size() - 1, left, right);
        return node.ans;
    }
    void update(int index, int value) {
        update(0, 0, a.size() - 1, index, value);
    }
};

int main() {
    int n, m; cin >> n >> m;
    vector<int> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];
    SegmentTree segmentTree = SegmentTree(a);
    cout << segmentTree.query(0, n - 1) << '\n'; // before ops
    while (m--) {
        int i, v;
        cin >> i >> v;
        segmentTree.update(i, v);
        cout << segmentTree.query(0, n - 1) << '\n'; // after
    }
}
```

Point Update, Range Sum (Array Style)

```
#include <bits/stdc++.h>
using namespace std;
#define _42 ios_base::sync_with_stdio(false); \
    cin.tie(NULL); cout.tie(NULL);

class SegmentTree {
private:
    vector<int> tree;
    int n;

    int query(int node, int left, int right,
              int qLeft, int qRight) {
        if (right < qLeft || left > qRight) return 0; // outside
        if (qLeft <= left && right <= qRight)
            return tree[node];                        // inside
        int mid = (left + right) / 2;
        return query(node*2+1, left, mid, qLeft, qRight)
             + query(node*2+2, mid+1, right, qLeft, qRight);
    }

    void update(int node, int left, int right,
                int index, int value) {
        if (left == right) {          // leaf node
            tree[node] = value;
            return;
        }
        int mid = (left + right) / 2;
        if (index > mid)
            update(node*2+2, mid+1, right, index, value);
        else
            update(node*2+1, left, mid, index, value);
        tree[node] = tree[node*2+1] + tree[node*2+2];
    }

public:
```

```
    SegmentTree(int n) {
        this->n = n;
        this->tree.assign(4 * n, 0);
    }
    void update(int index, int value) {
        return update(0, 0, n - 1, index, value);
    }
    int query(int left, int right) {
        if (left > right) return 0;
        return query(0, 0, n - 1, left, right);
    }
};

int main() {
    _42;
    int n; cin >> n;
    vector<int> array(2 * n);
    for (int i = 0; i < 2 * n; i++) cin >> array[i];
    SegmentTree segmentTree = SegmentTree(2 * n);
    vector<int> firstIndex(n + 1, -1);
    vector<int> answer(n + 1);
    for (int i = 0; i < 2 * n; i++) {
        int currNum = array[i];
        if (firstIndex[currNum] == -1) {
            firstIndex[currNum] = i;
        } else {
            int left = firstIndex[currNum];
            int right = i;
            answer[currNum] =
                segmentTree.query(left + 1, right - 1);
            segmentTree.update(left, 1);
        }
    }
    for (int i = 1; i < n + 1; i++) cout << answer[i] << ' ';
    return 0;
}
```

Counting Inversions (Fenwick-style via Segment Tree)

```
#include <bits/stdc++.h>
using namespace std;
#define _42 ios_base::sync_with_stdio(false); \
    cin.tie(NULL); cout.tie(NULL);

class SegmentTree {
private:
    vector<int> tree;
    vector<int> initialArray;
    vector<int> inversions;
    int n;

    int query(int node, int left, int right,
              int qLeft, int qRight) {
        if (right < qLeft || left > qRight) return 0; // no overlap
        if (qLeft <= left && right <= qRight)
            return tree[node];                    // full overlap
        int mid = (left + right) / 2;              // partial
        return query(node*2+1, left, mid, qLeft, qRight)
             + query(node*2+2, mid+1, right, qLeft, qRight);
    }
```

```
    void update(int node, int left, int right,
                int index, int value) {
        if (left == right) {
            tree[node] += value;
            return;
        }
        int mid = (left + right) / 2;
        if (index <= mid)
            update(node*2+1, left, mid, index, value);
        else
            update(node*2+2, mid+1, right, index, value);
        tree[node] = tree[node*2+1] + tree[node*2+2];
    }

public:
    SegmentTree(vector<int>& initialArray) {
        this->initialArray = initialArray;
        n = initialArray.size();
        inversions.assign(n, 0);
        tree.assign(4 * n, 0);
    }
    int query(int left, int right) {
        if (left > right) return 0;
        return query(0, 0, n - 1, left, right);
    }
    void update(int index, int value) {
        update(0, 0, n - 1, index, value);
    }
    vector<int> getInversions() {
        for (int i = 0; i < n; i++) {
            int value = initialArray[i] - 1;
            // count previous values greater than current
            inversions[i] = query(value + 1, n - 1);
            update(value, 1);          // mark current value seen
        }
        return inversions;
    }
};

int main() {
    _42;
    int n;
    cin >> n;
    vector<int> initialArray(n);
    for (int i = 0; i < n; i++) cin >> initialArray[i];
    SegmentTree segmentTree(initialArray);
    vector<int> inversions = segmentTree.getInversions();
    for (int x : inversions) cout << x << ' ';
    cout << '\n';
    return 0;
}
```

Range Update, Point Query

```
#include <bits/stdc++.h>
using namespace std;
#define _42 ios_base::sync_with_stdio(false); \
    cin.tie(NULL); cout.tie(NULL);
```

```
class SegmentTree {
private:
    vector<long long> tree;
    int n;

    long long query(int node, int left, int right, int index) {
        if (left == right) return tree[node];
        int mid = left + (right - left) / 2;
        if (index <= mid)
            return tree[node] + query(node*2+1, left, mid, index);
        else
            return tree[node] + query(node*2+2, mid+1, right, index);
    }

    void update(int node, int left, int right,
                int uLeft, int uRight, int value) {
        if (uRight < left || uLeft > right) return; // no overlap
        if (uLeft <= left && right <= uRight) {      // full overlap
            tree[node] += value;
            return;
        }
        int mid = left + (right - left) / 2;         // partial
        update(node*2+1, left, mid, uLeft, uRight, value);
        update(node*2+2, mid+1, right, uLeft, uRight, value);
    }

public:
    SegmentTree(int n) {
        this->n = n;
        tree.assign(n * 4, 0LL);
    }
    long long query(int index) {
        return query(0, 0, n - 1, index);
    }
    void update(int left, int right, int value) {
        if (left > right) return;
        update(0, 0, n - 1, left, right, value);
    }
};

void solve() {
    int n, m;
    if (!(cin >> n >> m)) return;
    SegmentTree segmentTree(n);
    while (m--) {
        int operation; cin >> operation;
        if (operation == 1) {
            int l, r, v;
            cin >> l >> r >> v;
            segmentTree.update(l, r - 1, v);
        } else if (operation == 2) {
            int index; cin >> index;
            cout << segmentTree.query(index) << '\n';
        }
    }
}

int main() {
    _42
    solve();
    return 0;
```

```
}
```

6 Combinatorics

GCD, LCM

```
// A x B = GCD(A, B) x LCM(A, B)
// LCM(A, B) = (A x B) / GCD(A, B)
long long lcm(long long a, long long b) {
    return (a / std::gcd(a, b)) * b;
}
long long gcd(long long a, long long b) {
    return b == 0 ? a : gcd(b, a % b);
}
```

Factorial Precomputation (n!, Modular Inverse Factorial)

```
const long long MOD = 1e9 + 7;
const int MAXF = 200005;
vector<long long> fact(MAXF), inv_fact(MAXF);

// Precomputes n! and the modular inverse of n! for all n < MAXF
// Requires the power(a, b, mod) fast-exponentiation function
void precompute_factorials() {
    fact[0] = 1;
    for (int i = 1; i < MAXF; i++)
        fact[i] = fact[i - 1] * i % MOD;
    // Fermat's Little Theorem: inverse(a) = a^(MOD-2) mod MOD
    // (MOD must be prime)
    inv_fact[MAXF - 1] = power(fact[MAXF - 1], MOD - 2, MOD);
    for (int i = MAXF - 2; i >= 0; i--)
        inv_fact[i] = inv_fact[i + 1] * (i + 1) % MOD;
}
```

Fast Power (Binary Exponentiation)

```
// Computes (a ^ b) % mod in O(log b)
long long power(long long a, long long b, long long mod) {
    a %= mod;
    long long res = 1;
    while (b > 0) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}
```

Modular Inverse & Division under Modulo

```
// Fermat's Little Theorem: a^(mod-2) is the inverse of a
// (mod is prime). Requires power(a, b, mod).
long long modInverse(long long a, long long mod) {
    return power(a, mod - 2, mod);
}
// (a / b) % mod == (a * inverse(b)) % mod
long long divMod(long long a, long long b, long long mod) {
    return (a % mod) * modInverse(b, mod) % mod;
}
```

```cpp
// Extended Euclidean version -- works for ANY modulus
// (not just primes)
long long extgcd(long long a, long long b,
                 long long &x, long long &y) {
    if (b == 0) {
        x = 1; y = 0;
        return a;
    }
    long long x1, y1;
    long long g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}
long long modInverseExtGcd(long long a, long long mod) {
    long long x, y;
    extgcd(a, mod, x, y);
    return ((x % mod) + mod) % mod;
}
```

nCr and nPr (Combinations & Permutations)

```cpp
// nCr(n, r): ways to choose r items out of n (order doesn't
// matter). Requires fact[] and inv_fact[] precomputed.
long long nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * inv_fact[r] % MOD * inv_fact[n - r] % MOD;
}
// nPr(n, r): ways to arrange r items out of n (order matters)
long long nPr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * inv_fact[n - r] % MOD;
}
```